# COMP2221 Systems Programming Summative Coursework Report

[Student ID]001257809

## 1 Summary of Approach & Solution Design

This project aims to build a small, fault-tolerant memory allocator which works entirely inside a small contiguous heap that is provided at initialization. My main design idea is to keep the allocators simple enough to reason about, but still add lightweight safety checks so that bit-flips or corrupted pointers would not crash the program. For the structure, the allocator uses an explicit free list and performs splitting and coalescing when blocks are allocated or freed. I used this structure because it is common in allocators like dlmalloc [1], and it is predictable and easier to analyse when adding fault-tolerance.

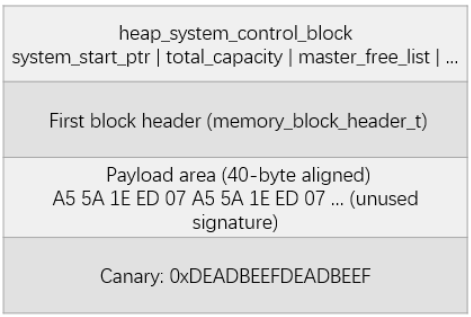



**Fig. 1.** Overall heap layout showing the system control block followed by memory blocks containing header, payload, and trailing canary.

As for metadata corruption detection, each block mirrored copies of some important fields, such as size and previous-block size. This idea is based on embedded-system reliability research, where redundancy is used to detect radiation-induced bit flips and soft errors before they propagate into system-level failures [2]. For overflow detection, I added a fixed canary word at the end of each payload. The mechanism is inspired by StackGuard-style buffer-overflow protection [3]: if the canary changes, the block is assumed to be corrupted and is quarantined.

The allocator also includes a fail-stop strategy based on the idea that it is safer to refuse further allocations than to rely on pointers that might be invalid. This aligns with the general fault-tolerance way discussed by Gray [4]. Any block that fails integrity checks is isolated so that it cannot corrupt the free list.

In conclusion, the design tries to combine well-understood allocation strategies [1] with lightweight reliability techniques [2][3], forming an allocator that behaves safely under normal operation and under simulated "radiation storm" errors [5].

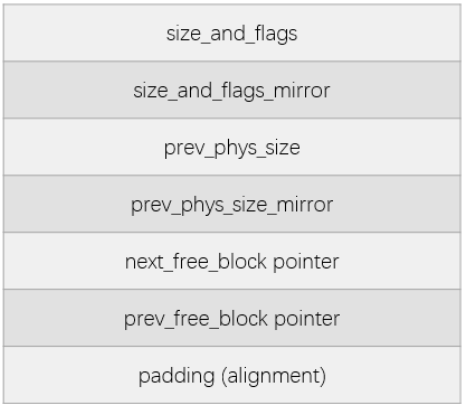



**Fig. 2.** Structure of memory block header, including mirrored fields for corruption detection and free-list pointers.

## 2 Analysis of Solution

For normal conditions, the allocator behaves like a straightforward explicit-free-list system. It walks the free list, uses first-fit, and supports splitting and two-way coalescing. These ideas are similar to the structures described in dlmalloc [1], and they help the allocator avoid unnecessary fragmentation. The correctness of the implementation relies heavily on checking pointer alignment, ensuring block sizes are valid, and updating mirror fields whenever metadata changes.

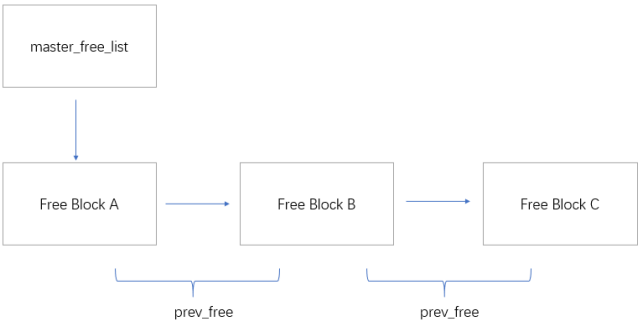



**Fig. 3.** Explicit doubly linked free list used by the allocator. Corrupted pointers are detected and offending blocks are quarantined.

The allocator uses two layers of protection for fault toler-

xxxx-xxxx/0x/$xx.00 © 200x IEEE Published by the IEEE Computer Society

ance. The mirrored fields help detect small metadata corruptions, such as single-bit flips. This follows reliability techniques described in embedded hardware parts [2][5]. The second layer is the payload canary, which detects out-of-bounds writes in the same way as StackGuard [3]. If either of these checks fails, the allocator quarantines the block immediately and removes it from the free list.

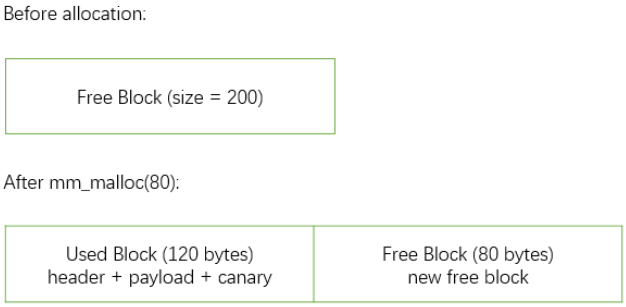



**Fig. 4.** Example of block splitting when a free block is larger than the requested size.

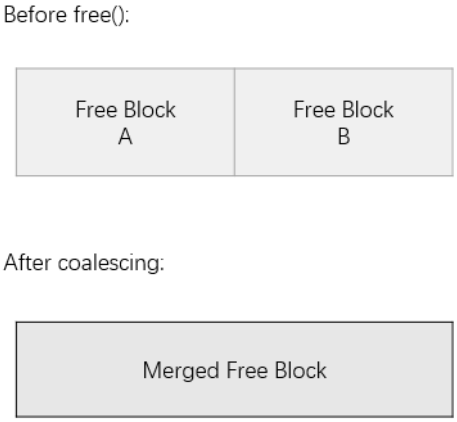



**Fig. 5.** Coalescing of adjacent free blocks using prev_phys_size and block headers.

| |
|---|
| Normal State: |
| [ Payload ..... ][ CANARY OK ] |
| Overflow write: |
| [ Payload ..... ][ CANARY CORRUPTED ] |
| Detection: |
| if (canary != 0xDEADBEEFDEADBEEF) → quarantine block |

**Fig. 6.** Canary-based overflow detection. Any modification of the canary indicates a buffer overrun and results in block quarantine.

One behaviour that stands out is how the allocator reacts during simulated storm tests. In my implementation, the free list is treated in a controversial way: if a pointer in the free list looks invalid, the allocator drops the entire list and moves into a "safe failure" mode. This matches Gray's fail-stop principle [4], where the system prioritises preventing further damage over availability. The result is that, during storms, all subsequent allocations fail—but

```
gjt@gjt-HP-Laptop-15s-du2xxx:~/summartive$ ./runme --seed 123 --storm 1 --size 8192
Using heap_size=8192, seed=123, storm_level=1
[PASS] mm_init                          ... OK
[PASS] allocate p1 (40)                 ... OK
[PASS] allocate p2 (80)                 ... OK
[PASS] allocate p3 (120)                ... OK
[PASS] free p2                          ... OK
[PASS] p1 alignment ok                  ... OK
[PASS] allocate p4                      ... OK
[PASS] mm_write/mm_read ok              ... OK
[PASS] mm_write out of bound            ... OK
[PASS] double free detection            ... OK

=== Storm test: level=1, seed=123 ===

Storm test summary:
  alloc ok      = 0
  alloc fail    = 313
  free ops      = 0
  write ok      = 0
  write fail    = 0
[PASS] storm test completed (no crash) ... OK

gjt@gjt-HP-Laptop-15s-du2xxx:~/summartive$ ./runme --seed 1   --storm 10 --size 65536
Using heap_size=65536, seed=1, storm_level=10
[PASS] mm_init                          ... OK
[PASS] allocate p1 (40)                 ... OK
[PASS] allocate p2 (80)                 ... OK
[PASS] allocate p3 (120)                ... OK
[PASS] free p2                          ... OK
[PASS] p1 alignment ok                  ... OK
[PASS] allocate p4                      ... OK
[PASS] mm_write/mm_read ok              ... OK
[PASS] mm_write out of bound            ... OK
[PASS] double free detection            ... OK

=== Storm test: level=10, seed=1 ===

Storm test summary:
  alloc ok      = 0
  alloc fail    = 366
  free ops      = 0
  write ok      = 0
  write fail    = 0
[PASS] storm test completed (no crash) ... OK

SUCCESS!
gjt@gjt-HP-Laptop-15s-du2xxx:~/summartive$ ./runme --seed 1   --storm 5 --size 65536
Using heap_size=65536, seed=1, storm_level=5
[PASS] mm_init                          ... OK
[PASS] allocate p1 (40)                 ... OK
[PASS] allocate p2 (80)                 ... OK
[PASS] allocate p3 (120)                ... OK
[PASS] free p2                          ... OK
[PASS] p1 alignment ok                  ... OK
[PASS] allocate p4                      ... OK
[PASS] mm_write/mm_read ok              ... OK
[PASS] mm_write out of bound            ... OK
[PASS] double free detection            ... OK

=== Storm test: level=5, seed=1 ===

Storm test summary:
  alloc ok      = 0
  alloc fail    = 320
  free ops      = 0
  write ok      = 0
  write fail    = 0
[PASS] storm test completed (no crash) ... OK

SUCCESS!
```

the allocator never crashes or dereferences invalid memory. The autograder's storm output is mostly "alloc fail", but the important part is that the program finishes cleanly without faults. This shows that the allocator's defensive checks are working as intended, although the strictness also means it does not achieve higher storm-resilience scores.

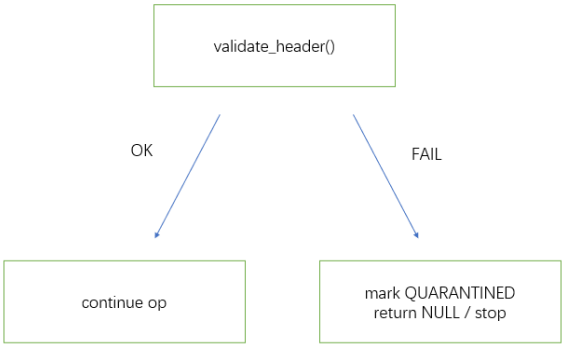
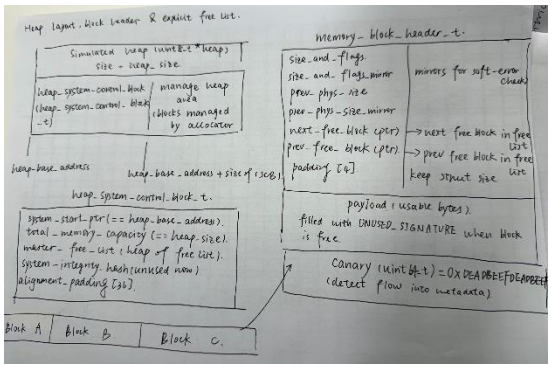



**Fig. 7.** Fail-stop quarantine logic inspired by Gray's error-containment principle. Corrupted blocks are isolated instead of reused.

## 3 Use of Generative AI, Tools, or other Resources

I used generative AI tools mainly to help me understand and verify certain design ideas, but not to generate the full implementation. For example, I asked for clarification about how a safe realloc should behave in corner cases, such as when a block is too small to expand or when canary checks fail. I also used AI to help compare different fault-tolerance strategies, including the fail-stop idea that inspired my quarantine behaviour.

As for the report, I use AI to beautify some words because I am afraid that my language was not professional enough. All allocator logic, including the design of metadata fields and free-list manipulation was written and tested manually. AI acted more as a support tool for reasoning rather than something that produced code directly.



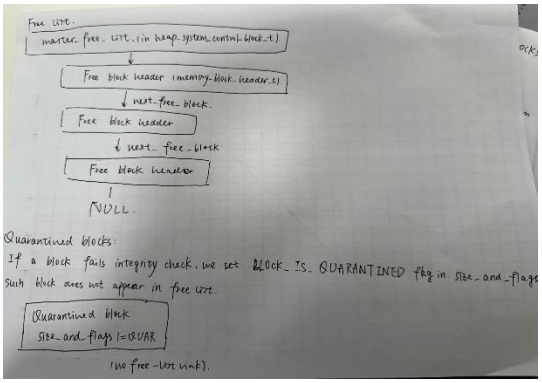




Fig 3.1 Overall heap layout and block organisation & Structure of a memory block header

Fig 3.2 Explicit free-list structure

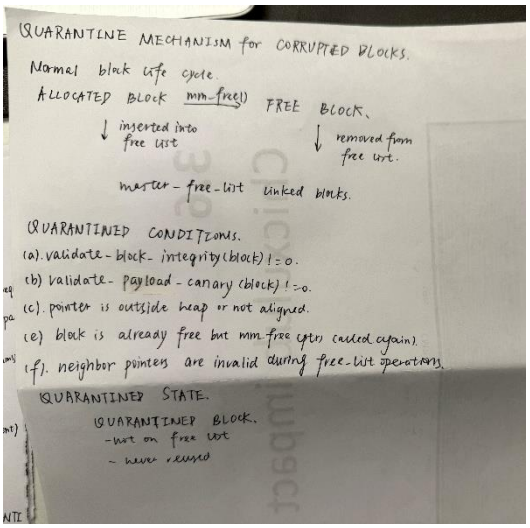



Fig 3.3 Quarantine mechanism

## 4 Additional Functionality

I implemented an additional function, mm_realloc, which was not required by the assignment. Its behaviour follows the conventional rules used by standard C library allocators: if the new size fits inside the current block, the pointer stays the same; otherwise, a new block is allocated, data is copied, and the old block is freed. This general pattern is consistent with typical realloc semantics described in standard C references [6].

What makes my mm_realloc notable is that it integrates smoothly with the allocator's fault-tolerance features. Before resizing, it performs the same mirror-field checks and canary verification used throughout the rest of the system [2][3]. If corruption is detected, the block is quarantined instead of resized, following the fail-stop rationale from [4]. This ensures that realloc does not introduce new integrity risks and demonstrates that the allocator's design can be extended safely.

Although optional, implementing mm_realloc helped me test the allocator in more situations. It also showed that the metadata layout was flexible enough to support new operations without breaking reliability.

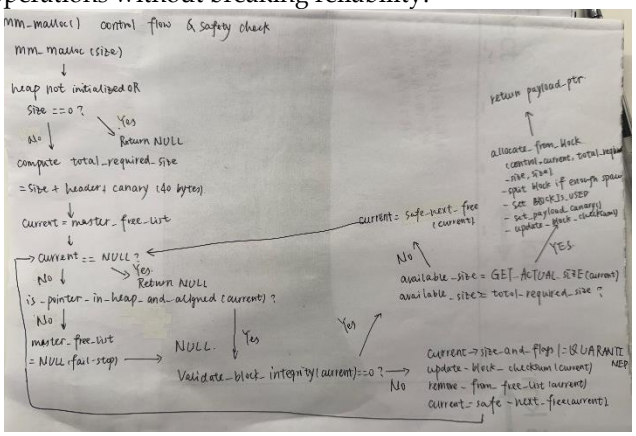



Fig 4.1 Control flow of mm_malloc()

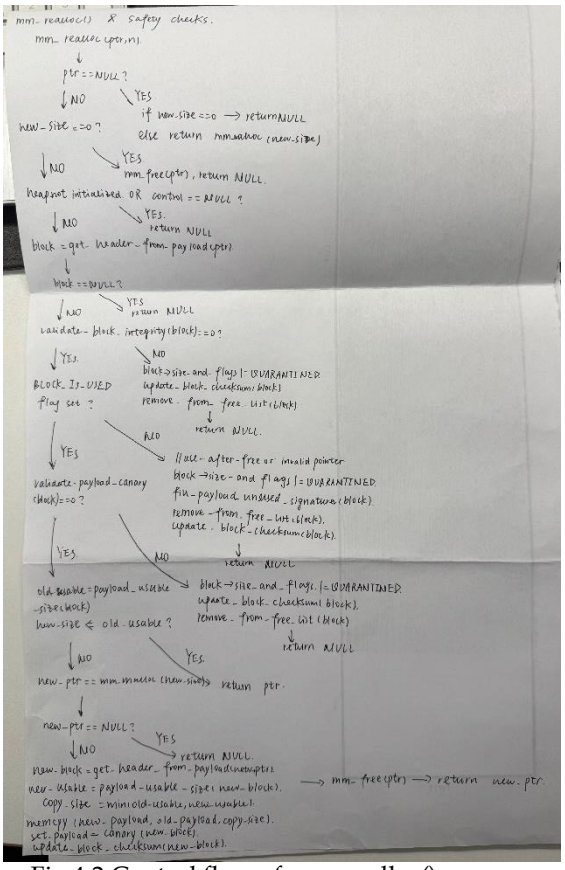



Fig 4.2 Control flow of mm_realloc()

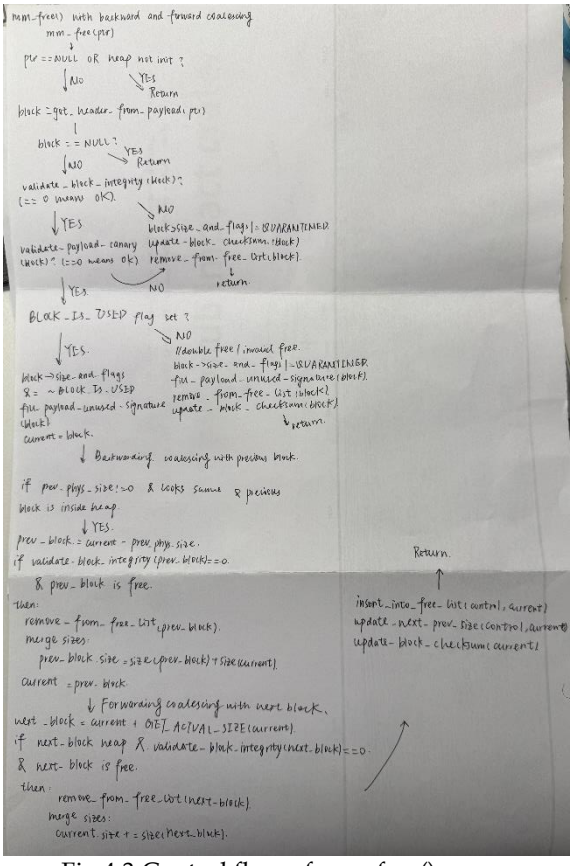



Fig 4.3 Control flow of mm_free()

## REFERENCES

[1] D. E. Knuth, Fundamental Algorithms, Addison-Wesley, 1997.

[2] R. Aalund and V. Paglioni, Enhancing Reliability in Embedded Systems Hardware: A Literature Survey, IEEE Access, 2025.

[3] C. Cowan et al., StackGuard: Automatic Adaptive Detection and Prevention of Buffer-Overflow Attacks, USENIX Security Symposium, 1998.

[4] J. Gray, Why Do Computers Stop and What Can Be Done About It?, 1986.

[5] R. C. Baumann, Radiation-Induced Soft Errors in Advanced Semiconductor Technologies, IEEE TDMR, 2005.

[6] ISO/IEC 9899:2018, Programming Languages—C Standard